

UNIVERSIDAD NACIONAL

Proyecto 1

Sistema de Alquiler de Vehículos

Documentación escrita

Curso: EIF206 - Programación III

Profesora: Kristel Duarte Pérez

Estudiantes:

Sebastián Arce Mejía

Justin Gamboa Badilla

Oscar Alvarado Méndez

**Fecha de entrega: 13 de
septiembre de 2026**

Tabla de contenido

Introducción	3
Descripción del proyecto	4
Funcionalidades principales	5
Justificación	6
Análisis comparativo del uso de Inteligencia Artificial	7
Conclusiones	7

Introducción:

En este documento se describe el desarrollo de un Sistema de Alquiler de Vehículos. El sistema fue creado usando Java, y en él se aplican conceptos clave de Programación Orientada a Objetos. Entre esos conceptos se incluyen herencia, colecciones, encapsulamiento y manejo de excepciones. También se usaron otros elementos importantes como la genética, aunque en este contexto se refiere a patrones de diseño o estructuras de datos específicas.

Este Sistema tiene como propósito aportar a la administración de una empresa dedicada al alquiler de motos, bicimotos y cuatriciclos. Por ello, este proyecto modela clientes, vehículos y alquileres, controlando la disponibilidad de la Flota: calculando costos, multas por atraso, depósitos y también este permitiría registrar la devolución de vehículos.

Fuera de las operaciones principales del alquiler, este proyecto incorpora un servicio de recomendaciones que analiza el presupuesto del cliente como la cantidad de días de alquiler y el tipo de vehículo que desea, esto para mostrarle alternativas disponibles y con las mejores opciones con estos filtros.

En lo siguiente, veremos cómo se presenta la descripción del proyecto y de su estructura, la justificación de la temática seleccionada, un análisis del uso de Inteligencia Artificial durante el desarrollo y las principales conclusiones obtenidas a partir de la experiencia.

Descripción del proyecto:

Este proyecto consiste en un sistema para administrar una flota de vehículos de alquiler. Con esta lógica se permite registrar y consultar clientes, como también gestionar distintos tipos de vehículos, crear alquileres, controlar cambios de estado y procesar devoluciones. Se hizo a partir de organizarla en paquetes para separar entidades del dominio y también la lógica del negocio, como sus interfaces y el almacenamiento temporal de información.

A continuación, se verá la estructura del código y su separación:

Paquete / componente	Elementos principales	Responsabilidad
model	Vehiculo, Moto, Bicimoto, Cuadraciclo, Cliente, Alquiler, EstadoVehiculo	Representa las entidades y estados principales del sistema.
interfaces	Alquilable	Define el contrato común para calcular el costo de un alquiler.
repository	Repositorio<T>	Administra colecciones dinámicas mediante una estructura genérica reutilizable.
service	VehiculoService, ClienteService, AlquilerService, RecomendacionService	Contiene validaciones y lógica de negocio.
JavaFX	HelloApplication, HelloController, hello-view.fxml	Contiene la base de la interfaz gráfica del proyecto.

La clase abstracta Vehiculo, concentra los datos comunes: placa, tarifa diaria, estado, marca. También implementa la interfaz Alquilable, en esta se logran especializar tres clases: Moto, bicimoto y cuadraciclo, con esto se logra aplicar herencia y se evita repetir atributos.

Cada subtipo incluye datos particulares: Moto tiene la cilindrada; Bicimoto indica el tipo de motor; y Cuadraciclo registra el tipo de tracción. El estado de cada unidad se muestra con EstadoVehiculo, lo que ayuda a saber si el vehículo está disponible o en uso.

Funcionalidades principales:

Gestión de clientes. El `ClienteService` permite registrar, buscar, listar, actualizar y eliminar clientes. Los datos que maneja la entidad `Cliente` son identificación, nombre, teléfono y correo. El servicio incluye validaciones para evitar que los datos obligatorios estén vacíos y para evitar registros duplicados por identificación.

Gestión de vehículos. `VehiculoService` permite registrar los vehículos de la flota, buscarlos por placa, listar todos los registros, mostrar solo los disponibles, actualizar información, cambiar el estado y eliminar unidades. La lógica evita registrar placas duplicadas y protege las operaciones que no sean consistentes con el estado del vehículo.

Registro de alquileres. `AlquilerService`, en este es donde se recibe el cliente, al vehículo, la fecha de inicio, la cantidad de días y el depósito de garantía. Antes de que se haga la registración de la operación, se valida primero que los datos estén correctos y que el vehículo este disponible. Una vez confirmado esto, el estado del vehículo cambia a alquilado y se le asigna un identificador consecutivo

Cálculo del costo. El costo base se obtiene mediante el método `calcularCostoAlquiler(int dias)` definido por el contrato `Alquilable` e implementado en `Vehiculo`. El cálculo se realiza multiplicando la tarifa diaria por la cantidad de días. La clase `Alquiler` guarda ese costo y el depósito de garantía, y su método `calcularTotal()` devuelve la suma de ambos valores.

Devolución y atraso. La fecha de devolución prevista se calcula sumando los días al inicio. Si la devolución ocurre después de esa fecha, el sistema determina los días de atraso y `AlquilerService` calcula una multa igual a una tarifa diaria por cada día adicional. El depósito de garantía se devuelve completo y la multa se maneja por separado. Al finalizar la devolución, el alquiler se marca como finalizado y el vehículo vuelve a estar `DISPONIBLE`.

Recomendaciones. `RecomendacionService` filtra los vehículos disponibles según la cantidad de días, el presupuesto máximo y el tipo solicitado. Los tipos permitidos son `MOTO`, `BICIMOTO`, `CUADRICICLO` o `TODOS`. Los resultados se ordenan de menor a mayor costo y el servicio puede mostrar la alternativa más económica que cumpla con los requisitos.

Programación genérica. `Repositorio<T>` usa `List<T>` y `ArrayList<T>` para almacenar objetos en memoria. Gracias al parámetro genérico `T`, la misma estructura se puede usar para clientes, vehículos y alquileres sin necesidad de crear repositorios diferentes para cada entidad.

Justificación

Se decidió crear un sistema para alquilar vehículos porque este tipo de sistema refleja una situación real. En este caso, el sistema debe gestionar recursos, clientes, fechas y pagos. Una empresa que alquila motos, bicimotos o cuadraciclos necesita saber siempre qué unidades están libres, qué unidades están en uso, a quién se les han entregado y cuándo deben regresar.

Hacer estos pasos a mano puede provocar errores. Por ejemplo, se podría asignar el mismo vehículo a dos clientes. También es posible calcular mal el costo del alquiler. Otra situación es no recordar cuándo un cliente debe devolver el vehículo. O no darse cuenta a tiempo si hay un retraso. Por eso, una aplicación que controle todas estas acciones puede ayudar a manejar mejor la flota y evitar problemas.

Desde el punto de vista académico, el tema es muy adecuado para aplicar los contenidos del curso. Los distintos tipos de vehículos comparten datos y acciones, pero también tienen características propias. Esto permitió usar una clase base abstracta y clases derivadas, aplicando herencia de forma natural. El proyecto también permitió usar una interfaz para definir un comportamiento común de cálculo. Se usaron colecciones dinámicas para almacenar registros. También se creó una clase genérica para reutilizar la misma estructura de almacenamiento. Además, se separaron los servicios para concentrar las reglas del negocio y las validaciones. Adicionalmente, la lógica de alquiler y devolución permitió trabajar con cambios de estado, fechas, cálculos y excepciones. El servicio de recomendaciones añadió un componente adicional de análisis al sistema. No solo registra información, sino que usa los datos disponibles para proponer opciones que se ajustan a las condiciones indicadas por el cliente.

Análisis comparativo del uso de Inteligencia Artificial

En el desarrollo del proyecto, ChatGPT se utilizó como ayuda para analizar problemas de programación. También se usó para organizar la estructura del proyecto. Además, ChatGPT ayudó a entender conceptos de Java y Programación Orientada a Objetos.

Una de las áreas donde ChatGPT fue más útil fue en la estructuración del proyecto. Ayudó a analizar cómo dividir las responsabilidades entre las clases del modelo, los servicios y el repositorio. Esto permitió evitar concentrar toda la lógica en una sola clase.

Otra parte importante fue la implementación de programación genérica mediante Repositorio<T>. El apoyo de IA permitió analizar cómo una sola clase podía manejar diferentes tipos de objetos usando List<T> y ArrayList<T>, en lugar de repetir una estructura de almacenamiento para cada entidad.

Las respuestas de la herramienta no se tomaron como soluciones automáticas. Fue necesario revisar el código, adaptarlo a la estructura del proyecto, comprobar que correspondiera con los contenidos vistos en clase y mantener una implementación que pudiera ser comprendida y explicada durante la defensa.

Conclusiones:

El desarrollo del Sistema de Alquiler de Vehículos ayudó a usar de forma práctica varios conceptos que se estudiaron en Programación III. La experiencia mostró que es importante repartir las responsabilidades del programa en clases y paquetes, para que cada componente tenga una tarea clara y el código sea más fácil de mantener.

La construcción de la jerarquía Vehiculo, Moto, Bicimoto y Cuadraciclo ayudó a entender mejor el uso de herencia y encapsulamiento. También la interfaz Alquilable mostró cómo definir un comportamiento común, y Repositorio<T> permitió usar programación genérica y reutilizar la misma estructura con diferentes tipos de objetos.

La lógica de alquileres y devoluciones fortaleció el manejo de estados, fechas, excepciones y cálculos. Se tuvo que comprobar que solo se alquilaran vehículos disponibles, registrar bien el costo y el depósito, calcular posibles atrasos y devolver la unidad al estado disponible tras terminar la operación.

El uso de la inteligencia artificial fue muy útil como herramienta de apoyo para poder analizar y lograr resolver los problemas presentados en el proyecto.

En conclusión, el proyecto permitió integrar conocimientos de Programación Orientada a Objetos, estructuras de datos, genéricos, validaciones y lógica de negocio dentro de una solución relacionada con un problema real. Además, contribuyó a fortalecer la capacidad de analizar código, organizar una aplicación y utilizar herramientas de IA de forma crítica.